

Relative path: data_gen/generate_logs.py

Filename: generate_logs.py Extension: .py

"""

OASIS v2 – Synthetic Data Generator

=====

Produces:

- sessions.jsonl model-facing Session records (NO label field)
- ground_truth.json session_id -> {"label": "normal"|"attack", "anomaly_type": ...}
 kept in a side channel, never merged into sessions.jsonl

Documented behavioral assumptions

- "Normal" is defined per entity_type, not globally:
 - * user: regular office-hours logins (with some spread), a stable small set of resources, one home geo, one habitual device.
 - * service_account: runs on a schedule (batch windows), touches a fixed set of DB/API resources, no geo variance (single DC).
 - * edge_device: near-constant heartbeat/sensor cadence, one fixed geo (its physical install site), one device fingerprint (its own firmware/MAC) that basically never changes under normal operation.
- Noise is Gaussian on continuous fields (hour-of-day, duration) and small-probability substitution on categorical fields (occasional off-list resource, occasional short session), so the "normal" population is not degenerately clean.
- Ground truth ("attack" vs "normal", and which of the 7 attack types) NEVER appears on a Session object. It only exists in ground_truth.json, keyed by session_id, so evaluation can be scored without any risk of the label leaking into a model-facing feature.

Attack taxonomy (7 types + 1 non-attack edge case)

See ATTACK_SIMULATION_LOGIC below for the exact simulation logic per type; this docstring lists what each represents and why the simulated fields make it detectable in principle, without hand-holding the model with the label.

1. brute_force – many failed auths, one source, short window.
2. impossible_travel – same entity, two geos, implausible time gap.
3. credential_stuffing – many entity_ids, few source_ips, high failure rate.
4. lateral_movement – unusual breadth: resources this entity has never touched, accessed out of habitual order.
5. low_and_slow – gradual buildup across several days: each single session looks only mildly odd, the trend doesn't.
6. privilege_escalation – a privileged action this entity type never performs appears mid-session.
7. device_spoofing – device_fingerprint changes without a plausible provisioning story (mismatched OS/MAC pattern).
8. insider_drift (EDGE CASE, label=normal) – entity's real behavior shifts (new hours / new device) gradually and *consistently* across many sessions. This must NOT be permanently flagged once the baseline profiler's decay catches up (see profiling/baseline_model.py).

"""

from __future__ import annotations

import json

import random

import uuid

from datetime import datetime, timedelta

from pathlib import Path

import numpy as np

RNG_SEED = 42

random.seed(RNG_SEED)

np.random.seed(RNG_SEED)

OUT_DIR = Path(__file__).resolve().parent.parent / "data"

OUT_DIR.mkdir(exist_ok=True)

SIM_START = datetime(2026, 6, 1, 0, 0, 0)

SIM_DAYS = 14

```

# Action / resource vocabularies per entity_type ("what normal means")
# -----
NORMAL_ACTIONS = {
    "user": ["login", "view_dashboard", "read_file", "download_report",
             "send_email", "browse_web", "view_profile", "update_settings",
             "logout"],
    "service_account": ["login", "api_call_read", "api_call_write",
                        "batch_job_start", "batch_job_end", "read_db",
                        "write_db", "logout"],
    "edge_device": ["connect", "heartbeat", "read_sensor", "write_actuator",
                    "firmware_check", "sync_data", "disconnect"],
}

PRIVILEGED_ACTIONS = ["grant_admin", "access_admin_panel",
                      "modify_permissions", "access_root_shell",
                      "disable_logging"]

RESOURCE_POOL = {
    "user": [f"file_share_{i}" for i in range(1, 6)] +
             [f"report_{i}" for i in range(1, 4)],
    "service_account": [f"db_table_{i}" for i in range(1, 6)] +
                       [f"api_endpoint_{i}" for i in range(1, 4)],
    "edge_device": [f"sensor_{i}" for i in range(1, 6)] +
                   [f"actuator_{i}" for i in range(1, 3)],
}

HIGH_VALUE_RESOURCE = "plc_controller_primary" # the one asset that matters

GEOS = ["Bengaluru,IN", "Mumbai,IN", "Singapore,SG", "Frankfurt,DE",
        "Ashburn,US", "Tokyo,JP", "Lagos,NG", "Sao_Paulo,BR"]

GEO_COORDS = {
    "Bengaluru,IN": (12.9716, 77.5946),
    "Mumbai,IN": (19.0760, 72.8777),
    "Singapore,SG": (1.3521, 103.8198),
    "Frankfurt,DE": (50.1109, 8.6821),
    "Ashburn,US": (39.0438, -77.4874),
    "Tokyo,JP": (35.6762, 139.6503),
    "Lagos,NG": (6.5244, 3.3792),
    "Sao_Paulo,BR": (-23.5505, -46.6333),
}

MAX_PLAUSIBLE_SPEED_KMH = 1000.0

AUTH_METHODS = ["password", "token", "certificate", "biometric"]

def new_id(prefix: str) -> str:
    return f"{prefix}_{uuid.uuid4().hex[:10]}"

def haversine_km(geo_a: str, geo_b: str) -> float:
    lat1, lon1 = GEO_COORDS[geo_a]
    lat2, lon2 = GEO_COORDS[geo_b]
    radius_km = 6371.0
    phi1, phi2 = np.radians(lat1), np.radians(lat2)
    dphi = np.radians(lat2 - lat1)
    dlambd = np.radians(lon2 - lon1)
    a = np.sin(dphi / 2) ** 2 + np.cos(phi1) * np.cos(phi2) * np.sin(dlambd / 2) ** 2
    return float(2 * radius_km * np.arcsin(np.sqrt(a)))

def sample_geo_for_session(entity, session_start_time, last_session_end, last_geo, noise_prob=0.01):
    if last_geo is None or random.random() > noise_prob:
        return entity["home_geo"]
    candidate = random.choice([g for g in GEOS if g != last_geo])
    if last_session_end is not None:
        hours_gap = (session_start_time - last_session_end).total_seconds() / 3600
        if hours_gap > 0:
            distance_km = haversine_km(last_geo, candidate)
            if (distance_km / hours_gap) > MAX_PLAUSIBLE_SPEED_KMH:
                return entity["home_geo"]
    return candidate

# -----
# Entity roster (25 entities: 12 user, 7 service_account, 6 edge_device)

```

```

# -----
def build_entities():
    entities = []
    for i in range(12):
        entities.append({
            "entity_id": f"user_{i:02d}",
            "entity_type": "user",
            "home_geo": random.choice(GEOS),
            "home_device": new_id("dev"),
            "typical_hour_mean": random.choice([9, 10, 14]),
            "typical_hour_std": 1.5,
            "typical_resources": random.sample(RESOURCE_POOL["user"], k=3),
            "typical_session_len": random.randint(4, 8),
            "typical_duration_mean": random.uniform(300, 1200),
        })
    for i in range(7):
        entities.append({
            "entity_id": f"svc_{i:02d}",
            "entity_type": "service_account",
            "home_geo": "Bengaluru,IN", # single DC, no geo variance
            "home_device": new_id("dev"),
            "typical_hour_mean": random.choice([1, 2, 3]), # batch windows
            "typical_hour_std": 0.5,
            "typical_resources": random.sample(RESOURCE_POOL["service_account"], k=3),
            "typical_session_len": random.randint(3, 6),
            "typical_duration_mean": random.uniform(60, 300),
        })
    for i in range(6):
        entities.append({
            "entity_id": f"edge_{i:02d}",
            "entity_type": "edge_device",
            "home_geo": random.choice(GEOS),
            "home_device": new_id("dev"), # own firmware/MAC id, near-static
            "typical_hour_mean": None, # heartbeats are ~constant, not diurnal
            "typical_hour_std": None,
            "typical_resources": random.sample(RESOURCE_POOL["edge_device"], k=2),
            "typical_session_len": random.randint(3, 5),
            "typical_duration_mean": random.uniform(30, 120),
        })
    # the one edge_device that also touches the high-value PLC asset
    entities[-1]["typical_resources"].append(HIGH_VALUE_RESOURCE)
    return entities

def make_session_skeleton(entity, start_time, duration_override=None,
                          command_sequence=None, resources=None,
                          auth_failures=0, geo=None, device_fp=None,
                          source_ip=None):
    duration = duration_override or max(
        10, np.random.normal(entity["typical_duration_mean"], 60))
    end_time = start_time + timedelta(seconds=duration)
    seq = command_sequence or _sample_normal_sequence(entity)
    res = resources if resources is not None else list(
        a for a in entity["typical_resources"]
        if random.random() < 0.8
    ) or [random.choice(entity["typical_resources"])]
    geo = geo or entity["home_geo"]
    device_fp = device_fp or entity["home_device"]
    src_ip = source_ip or f"10.{random.randint(0,255)}.{random.randint(0,255)}.{random.randint(1,254)}"
    return {
        "session_id": new_id("sess"),
        "entity_id": entity["entity_id"],
        "entity_type": entity["entity_type"],
        "start_time": start_time.isoformat(),
        "end_time": end_time.isoformat(),
        "session_duration_sec": round(duration, 1),
        "command_sequence": seq,
        "resources_touched": res,
        "auth_failures": auth_failures,
        "source_ips": [src_ip],
        "geo_locations": [geo],
        "device_fingerprint": device_fp,
        "is_new_device": device_fp != entity["home_device"],
    }

```

```

def _sample_normal_sequence(entity):
    vocab = NORMAL_ACTIONS[entity["entity_type"]]
    n = max(2, int(np.random.normal(entity["typical_session_len"], 1)))
    body = [random.choice(vocab[1:-1]) for _ in range(max(0, n - 2))]
    return [vocab[0], *body, vocab[-1]]

def _random_start_time(day_offset_range=(0, SIM_DAYS), entity=None):
    day = random.uniform(*day_offset_range)
    if entity and entity["typical_hour_mean"] is not None:
        hour = np.clip(np.random.normal(entity["typical_hour_mean"],
                                         entity["typical_hour_std"]), 0, 23.9)
    else:
        hour = random.uniform(0, 24)
    return SIM_START + timedelta(days=day, hours=hour) - timedelta(
        hours=(SIM_START + timedelta(days=day)).hour)

# -----
# Normal baseline generation
# -----
def generate_normal_sessions(entities, sessions_per_entity_per_day=3.5):
    sessions = []
    for entity in entities:
        n_sessions = int(SIM_DAYS * sessions_per_entity_per_day * random.uniform(0.7, 1.3))
        start_times = sorted(_random_start_time(entity=entity) for _ in range(n_sessions))
        last_geo = None
        last_end = None
        for start in start_times:
            geo = sample_geo_for_session(entity, start, last_end, last_geo)
            session = make_session_skeleton(entity, start, geo=geo)
            sessions.append(session)
            last_geo = session["geo_locations"][0] if session["geo_locations"] else None
            last_end = datetime.fromisoformat(session["end_time"])
    return sessions

# -----
# Attack taxonomy – one injector function per type. Each returns
# (list_of_sessions, list_of_ground_truth_records).
# -----
def inject_brute_force(entities, n_incidents=16):
    sessions, gt = [], []
    users = [e for e in entities if e["entity_type"] == "user"]
    for _ in range(n_incidents):
        entity = random.choice(users)
        start = _random_start_time()
        s = make_session_skeleton(
            entity, start,
            duration_override=random.uniform(20, 90),
            command_sequence=["login"] * random.randint(6, 12) + ["logout"],
            resources=[],
            auth_failures=random.randint(8, 15),
            source_ip=f"185.{random.randint(0,255)}.{random.randint(0,255)}.1",
        )
        sessions.append(s)
        gt.append({"session_id": s["session_id"], "label": "attack",
                  "anomaly_type": "brute_force"})
    return sessions, gt

def inject_impossible_travel(entities, n_incidents=8):
    sessions, gt = [], []
    users = [e for e in entities if e["entity_type"] == "user"]
    for _ in range(n_incidents):
        entity = random.choice(users)
        t1 = _random_start_time()
        far_geo = random.choice([g for g in GEOS if g != entity["home_geo"]])
        s1 = make_session_skeleton(entity, t1, geo=entity["home_geo"])
        # second login from a far geo within an implausible gap (30-90 min)
        t2 = t1 + timedelta(minutes=random.randint(30, 90))
        s2 = make_session_skeleton(entity, t2, geo=far_geo)
        sessions += [s1, s2]
        gt.append({"session_id": s1["session_id"], "label": "normal",
                  "anomaly_type": "normal"})
        gt.append({"session_id": s2["session_id"], "label": "attack",

```

```

        "anomaly_type": "impossible_travel"})
return sessions, gt

def inject_credential_stuffing(entities, n_incidents=1, batch_size=16):
    sessions, gt = [], []
    for _ in range(n_incidents):
        start = _random_start_time()
        shared_ip = f"193.{random.randint(0,255)}.{random.randint(0,255)}.9"
        targets = random.sample(entities, k=min(batch_size, len(entities)))
        for entity in targets:
            s = make_session_skeleton(
                entity, start + timedelta(seconds=random.randint(0, 120)),
                duration_override=random.uniform(5, 20),
                command_sequence=["login"],
                resources=[],
                auth_failures=random.choice([1, 1, 2, 3]),
                source_ip=shared_ip,
            )
            sessions.append(s)
            gt.append({"session_id": s["session_id"], "label": "attack",
                      "anomaly_type": "credential_stuffing"})
    return sessions, gt

def inject_lateral_movement(entities, n_incidents=16):
    sessions, gt = [], []
    svc = [e for e in entities if e["entity_type"] == "service_account"]
    for _ in range(n_incidents):
        entity = random.choice(svc)
        start = _random_start_time()
        # breadth: touches resources across every pool, never-before-seen,
        # out of habitual order (read -> write -> admin-adjacent)
        broad_resources = (RESOURCE_POOL["service_account"] +
                           RESOURCE_POOL["user"][:2] + [HIGH_VALUE_RESOURCE])
        seq = ["login", "read_db", "api_call_read", "api_call_write",
              "write_db", "modify_permissions", "logout"]
        s = make_session_skeleton(
            entity, start, command_sequence=seq, resources=broad_resources)
        sessions.append(s)
        gt.append({"session_id": s["session_id"], "label": "attack",
                  "anomaly_type": "lateral_movement"})
    return sessions, gt

def inject_low_and_slow(entities, n_incidents=3):
    sessions, gt = [], []
    users = [e for e in entities if e["entity_type"] == "user"]
    for _ in range(n_incidents):
        entity = random.choice(users)
        # gradual buildup over the sim window: each session touches one more
        # off-profile resource than the last, individually mild, and each one
        # happens in a quieter window than the entity's normal schedule.
        extra_pool = [r for r in RESOURCE_POOL["user"] + RESOURCE_POOL["service_account"]
                      if r not in entity["typical_resources"]]
        n_steps = 5
        attack_hour = ((entity["typical_hour_mean"] or 10) + 8) % 24
        for i in range(n_steps):
            day_frac = i / n_steps * SIM_DAYS
            start = SIM_START + timedelta(days=day_frac, hours=attack_hour)
            res = list(entity["typical_resources"]) + extra_pool[: i + 1]
            s = make_session_skeleton(entity, start, resources=res)
            sessions.append(s)
            gt.append({"session_id": s["session_id"], "label": "attack",
                      "anomaly_type": "low_and_slow"})
    return sessions, gt

def inject_privilege_escalation(entities, n_incidents=16):
    sessions, gt = [], []
    non_edge = [e for e in entities if e["entity_type"] != "edge_device"]
    for _ in range(n_incidents):
        entity = random.choice(non_edge)
        start = _random_start_time()
        base = NORMAL_ACTIONS[entity["entity_type"]]
        seq = [base[0], random.choice(base[1:-1]),

```

```

        random.choice(PRIVILEGED_ACTIONS), base[-1]]
    s = make_session_skeleton(entity, start, command_sequence=seq)
    sessions.append(s)
    gt.append({"session_id": s["session_id"], "label": "attack",
              "anomaly_type": "privilege_escalation"})
return sessions, gt

def inject_device_spoofing(entities, n_incidents=16):
    sessions, gt = [], []
    for _ in range(n_incidents):
        entity = random.choice(entities)
        start = _random_start_time()
        spoofed_fp = new_id("dev") # unrelated fingerprint, no provisioning story
        s = make_session_skeleton(entity, start, device_fp=spoofed_fp)
        sessions.append(s)
        gt.append({"session_id": s["session_id"], "label": "attack",
                  "anomaly_type": "device_spoofing"})
    return sessions, gt

def inject_insider_drift(entities, n_incidents=1):
    """EDGE CASE – not an attack. One entity's real behavior legitimately
    shifts (new work hours + new device) gradually and consistently across
    the back half of the sim window. Ground truth label stays 'normal'.
    This is what profiling/baseline_model.py's decay must NOT keep flagging."""
    sessions, gt = [], []
    users = [e for e in entities if e["entity_type"] == "user"]
    for _ in range(n_incidents):
        entity = random.choice(users)
        new_hour = (entity["typical_hour_mean"] + 8) % 24 # e.g. 10am -> 6pm shift
        new_device = new_id("dev") # e.g. new laptop, consistently used from here on
        n_steps = 22
        for i in range(n_steps):
            day_frac = SIM_DAYS * 0.4 + i * (SIM_DAYS * 0.6 / n_steps)
            start = SIM_START + timedelta(days=day_frac, hours=new_hour)
            s = make_session_skeleton(entity, start, device_fp=new_device)
            sessions.append(s)
            gt.append({"session_id": s["session_id"], "label": "normal",
                      "anomaly_type": "insider_drift"})
        entity["_drift_entity"] = True
    return sessions, gt

ATTACK_SIMULATION_LOGIC = {
    "brute_force": inject_brute_force,
    "impossible_travel": inject_impossible_travel,
    "credential_stuffing": inject_credential_stuffing,
    "lateral_movement": inject_lateral_movement,
    "low_and_slow": inject_low_and_slow,
    "privilege_escalation": inject_privilege_escalation,
    "device_spoofing": inject_device_spoofing,
    "insider_drift": inject_insider_drift, # edge case, label=normal
}

def main():
    entities = build_entities()
    normal_sessions = generate_normal_sessions(entities)

    all_sessions = list(normal_sessions)
    ground_truth = {s["session_id"]: {"label": "normal", "anomaly_type": "normal"}
                    for s in normal_sessions}

    for name, fn in ATTACK_SIMULATION_LOGIC.items():
        new_sessions, gt_records = fn(entities)
        all_sessions += new_sessions
        for rec in gt_records:
            ground_truth[rec["session_id"]] = {"label": rec["label"],
                                                "anomaly_type": rec["anomaly_type"]}
    print(f" injected {name:22s} -> {len(new_sessions):3d} sessions")

    all_sessions.sort(key=lambda s: s["start_time"])

    sessions_path = OUT_DIR / "sessions.jsonl"
    with open(sessions_path, "w") as f:

```

```

    for s in all_sessions:
        f.write(json.dumps(s) + "\n")

gt_path = OUT_DIR / "ground_truth.json"
with open(gt_path, "w") as f:
    json.dump(ground_truth, f, indent=2)

entities_path = OUT_DIR / "entities.json"
with open(entities_path, "w") as f:
    json.dump([{"k": v for k, v in e.items() if not k.startswith("_")}
               for e in entities], f, indent=2)

n_attack = sum(1 for v in ground_truth.values() if v["label"] == "attack")
print(f"\nWrote {len(all_sessions)} sessions -> {sessions_path}")
print(f"  normal: {len(all_sessions) - n_attack}  attack: {n_attack}  "
      f"({n_attack/len(all_sessions):.1%} of sessions are attacks)")
print(f"Ground truth -> {gt_path} (side channel, never merged into sessions.jsonl)")

if __name__ == "__main__":
    main()

```